

Activity 2 : Compare Backend, Database and Authentication Options

2.1 Backend Framework Comparison

Criteria	Node.js / NestJS	Python / FastAPI	Go
Development Speed	High : shares JS/TS with frontend	High : concise, auto generated docs	Medium : more boilerplate
Real-time Support	Excellent (event loop, Socket.io)	Good (async, WebSockets)	Excellent (goroutines)
AI/ML Fit	Medium : must call out to Python/cloud services	Very High : native to TensorFlow/PyTorch	Low : limited ML ecosystem
Ecosystem / Hiring Pool	Very Large (npm, JS talent)	Large (Python, data science teams)	Medium (smaller, specialised talent)
Performance	Medium High	Medium High	Very High
Team Fit for FitFlow	Best : single language across RN frontend + backend	Good for the AI microservice specifically	Best for narrow, high throughput infra services

Table 2.1: Backend framework comparison

Node.js with NestJS/Express is best suited as the application backend because it lets the mid sized team share TypeScript types, validation logic and tooling with the React Native frontend and its event driven model is a natural fit for the apps real time social and notification features. Python/FastAPI remains valuable as the implementation language for the AI microservice, where its native ML ecosystem outweighs the cost of a second language confined to a single well isolated service. Go is not recommended as the backend given FitFlows need for rapid, business logic heavy CRUD development but could be considered later for a narrow high throughput component (, for example a dedicated notification fan out service) if scale demands it.

2.2 Database Comparison

Criteria	PostgreSQL	MongoDB	Firebase (Firestore/RTDB)	DynamoDB
Scalability	High (replicas, sharding via Citus)	High (native sharding)	Automatic, fully managed	Extremely High (AWS managed)

Criteria	PostgreSQL	MongoDB	Firebase (Firestore/RTDB)	DynamoDB
Query Performance	High for relational/complex queries	Good for document reads; weak joins	Excellent for simple real time queries	Very fast for key value access
Health Data Handling	Strong : ACID, row level security, mature encryption	Flexible but needs strict schema validation	Configurable for GDPR; HIPAA needs BAA	HIPAA eligible with AWS BAA
Best Fit in FitFlow	Users, workouts, subscriptions, health records	Flexible workout-template content	Social feed, chat, live updates	High throughput logs if AWS native

Table 2.2: Database options compared for scalability, query performance, and health-data handling

A polyglot persistence approach is recommended: PostgreSQL as the system of record for user profiles, subscriptions, and structured health/fitness data, where relational integrity, ACID guarantees, and mature row level security tooling matter most for compliance; and Firebase Firestore/Realtime Database for the social feed, chat, and live notifications, where its real time listeners and automatic scaling minimise operational overhead for a mid sized team and integrate natively with the React Native + Firebase ecosystem already used for authentication and push notifications.

2.3 Authentication and Authorization Comparison

Criteria	Firebase Auth	AWS Cognito	Auth0	Supabase
Integration Effort	Lowest : native to rest of Firebase stack	Medium : AWS ecosystem	Low Medium excellent SDKs	Low : native to PostgreSQL
Compliance	GDPR ready; HIPAA via GCP BAA	HIPAA : eligible with BAA	Strong : HIPAA/GDPR add-ons, enterprise-grade	GDPR ready; smaller compliance track record
Real-time / RBAC	Basic RBAC, pairs well with Firestore rules	Strong federation (SAML/OIDC)	Best in class fine : grained RBAC	Row level security tied to Postgres
Cost (mid-sized team)	Very Low : generous free tier	Low Medium	Higher per MAU at scale	Low

Table 2.3: Authentication and authorization options compared

2.4 Recommended Backend Combination

The recommended combination is Node.js/Express (NestJS structured) as the core backend, PostgreSQL for structured/compliance sensitive data, Firebase (Firestore/RTDB) for real time social and notification features, and Firebase Auth for authentication. This combination minimises vendor sprawl and integration cost for a mid sized team, since the real time database, push notifications, and authentication are all part of the same managed Firebase platform already required for the social features, while PostgreSQL is reserved specifically for the health and subscription data where relational integrity and fine grained access control matter most for HIPAA/GDPR compliance. This is justified further, with weighted scoring, in Activity 3.